2026 IEEE International Conference on Recent Advances in Electronics, Communication, Computing, Automation and Power (RECCAP) | 979-8-3315-4540-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/RECCAP69368.2026.11723378

Dynamic Modeling and Control of a 3-DOF Robotic Arm for Trajectory Tracking using PID and MPC

Tony K John
Dept. of Electrical & Electronics Engg.
TKM College of Engineering, Kollam
tonykjohn@outlook.com

Dr. Resmi R
Dept. of Electrical & Electronics Engg.
TKM College of Engineering, Kollam
resmijaidev@tkmce.ac.in

Dr. Chinnu Jacob
Centre for Artificial Intelligence
TKM College of Engineering, Kollam
chinnujacob@tkmce.ac.in

Najma Farsana
Dept. of Electrical & Electronics Engg.
TKM College of Engineering, Kollam
APJ Abdul Kalam Technological University
sananaisam27@gmail.com

Dr. Fousia M Shamsudeen
Dept. of Computer Applications
TKM College of Engineering, Kollam
fousiasahala@tkmce.ac.in

***Abstract*—This paper presents the modelling, control and simulation of a three degree of freedom(DOF) robotic arm to achieve precise trajectory tracking and improved dynamic performance. The arm is built to resemble the motion of a human arm, with links and joints connected in a way that allows flexible movement. Both the kinematics and dynamics of the system have been analyzed in detail. Forward and inverse kinematics are used to relate joint variables to the end-effector's position and orientation, while the dynamics is derived through the Euler–Lagrange method, which captures the effects of mass, inertia, Coriolis, centrifugal, and gravitational forces. Two distinct control strategies are employed to achieve precise trajectory tracking. The first is a Proportional–Integral–Derivative (PID) controller, chosen for its simplicity and popularity in engineering practice. Even though PID exhibits satisfactory performance, its constraints become evident when addressing the nonlinear and coupled dynamics of the robotic arm. To overcome these challenges, a Model Predictive Control (MPC) approach is adopted. MPC provides a more systematic framework, with the ability to include system constraints and anticipate future system behavior. The controllers are applied to a three-DOF arm, where MPC shows clear improvements in tracking accuracy and disturbance rejection compared to PID. The results demonstrate that MPC not only scales well to more complex systems but also delivers more reliable and efficient control than conventional methods.**

***Index Terms*—Robotic Arm, 3 DOF Manipulator, Kinematics, Trajectory Tracking, PID controller, Model Predictive Control.**

I. Introduction

Robotic manipulators, or robotic arms, are key elements of modern automation due to their ability to replicate human-arm functionality with high precision, repeatability, and speed. They are widely used in sectors such as manufacturing, healthcare, aerospace, and defense, particularly for tasks requiring accuracy, endurance, or for operating in hazardous environments [1]. As Industry 4.0 is continually growing, the need for intelligent and autonomous manipulators continues to increase, highlighting the importance of efficient modeling and control strategies [2].

Mathematical modeling is essential for the control and simulation of robotic manipulators, which consist of rigid links connected by revolute or prismatic joints that define their degrees of freedom. Modeling includes kinematics, relating joint variables to end-effector motion, and dynamics, describing how forces and torques generate movement [3]. Dynamic models which are derived through Euler–Lagrange or Newton–Euler methods capture the nonlinear and coupled nature of manipulators, and accurate modeling is crucial for achieving reliable control performance [11].

Among classical control methods, the Proportional Integral Derivative (PID) controller is widely used in industrial robotics due to its simplicity and reliable performance under nominal conditions [5]. Although effective for many tracking and positioning tasks, performance of PID controller is not satisfactory in the presence of nonlinear dynamics, coupling effects, or parameter variations. These limitations make PID less suitable for modern manipulators operating in dynamic and uncertain environments [6].

Several alternative control strategies also have been investigated. Fuzzy Logic Control (FLC) offers robustness without requiring an exact model, yet tuning is often empirical [8]. Sliding Mode Control (SMC) is robust to disturbances but suffers from chattering, which can stress actuators. More recently, Laguerre-based MPC formulations have been introduced to reduce computational load and improve real-time feasibility in distributed and constrained settings [17], [18]. These limitations and advances motivate the adoption of nonlinear MPC, which can jointly address optimality, robustness, and constraint handling.

Model Predictive Control (MPC) is an advanced strategy that computes control inputs by solving a finite-horizon optimization problem while explicitly accounting for system dynamics and constraints [9]. MPC predicts future system behavior and adjusts inputs proactively, which enables systematic handling of multivariable interactions, nonlinearities, and input or state limits [10]. Several studies show its advantages over traditional approaches. For example, comparisons with linear optimal controllers demonstrate improved tracking and

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.

constraint enforcement, while MPC has also been shown to outperform PID control under uncertainties and disturbances [6].

Despite significant progress in robotic manipulator control, several challenges remain in achieving accurate trajectory tracking for spatial manipulators with nonlinear coupled dynamics. Many existing studies focus on planar manipulators or rely on simplified linearized models for controller design. Even though model predictive control has been investigated for robotic systems, its application using nonlinear dynamic prediction with accurate numerical integration for spatial manipulators remains relatively less explored. These limitations motivate the development of a modeling and control framework capable of handling nonlinear manipulator dynamics while enabling accurate trajectory tracking in three-dimensional space.

The proposed work focuses on the modeling and control of a spatial three-degree-of-freedom (3-DOF) robotic manipulator, providing a structured framework for trajectory tracking using classical and predictive control methods. A complete kinematic and dynamic model is first derived using the Euler–Lagrange formulation, capturing nonlinear effects such as inertia, Coriolis, centrifugal, and gravity forces. A nonlinear model predictive control (NMPC) scheme is then developed, where system dynamics are discretized using a fourth-order Runge–Kutta (RK4) method for accurate state prediction without linearization. Finally, the performance of the proposed NMPC approach is compared with classical PID control through MATLAB simulations of circular trajectory tracking, demonstrating improved tracking accuracy and smoother control inputs with NMPC.

II. Modeling of a 3-DOF Robotic Arm

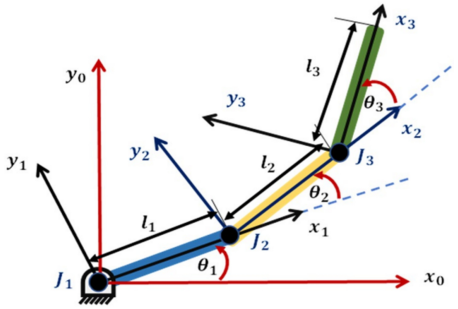


Fig. 1: Schematic diagram of the 3-DOF spatial robotic arm

This section presents the modeling of a three Degrees-of-Freedom (DOF) spatial robotic arm. As shown in Fig. 1, the 3-DOF spatial manipulator consists of three revolute joints.

Kinematics describes the motion of the arm in terms of positions and velocities without reference to the forces involved, whereas dynamics relates these motions to the applied torques and external forces. The Euler–Lagrange formulation is used to obtain the system equations, which later form the basis for trajectory tracking and control design.

A. Kinematic Modeling

Forward kinematics: Using the link geometry of Fig. 1, the end–effector position

$$\mathbf{p} = \mathbf{r}_1 + \mathbf{r}_2 + \mathbf{r}_3. \tag{1}$$

a) Link 1: The local vector of Link 1 is

$$\mathbf{r}_1^{(local)} = \begin{bmatrix} L_1 \\ 0 \\ 0 \end{bmatrix}. \tag{2}$$

After rotation by θ_1 about the global z–axis,

$$\mathbf{r}_1 = \mathbf{R}_z(\theta_1)\mathbf{r}_1^{(local)} = \begin{bmatrix} L_1\cos\theta_1 \\ L_1\sin\theta_1 \\ 0 \end{bmatrix}. \tag{3}$$

b) Link 2: After rotation by θ_2 about the local y–axis, the local vector of Link 2 is

$$\mathbf{r}_2^{(local)} = \begin{bmatrix} L_2\cos\theta_2 \\ 0 \\ L_2\sin\theta_2 \end{bmatrix}. \tag{4}$$

Transforming this vector into the base frame using $\mathbf{R}_z(\theta_1)$,

$$\mathbf{r}_2 = \begin{bmatrix} L_2\cos\theta_1\cos\theta_2 \\ L_2\sin\theta_1\cos\theta_2 \\ L_2\sin\theta_2 \end{bmatrix}. \tag{5}$$

c) Link 3: Defining the total bending angle $\theta_{23} = \theta_2 + \theta_3$, the local vector of Link 3 is

$$\mathbf{r}_3^{(local)} = \begin{bmatrix} L_3\cos(\theta_{23}) \\ 0 \\ L_3\sin(\theta_{23}) \end{bmatrix}. \tag{6}$$

Transforming into the base frame yields

$$\mathbf{r}_3 = \begin{bmatrix} L_3\cos\theta_1\cos(\theta_2+\theta_3) \\ L_3\sin\theta_1\cos(\theta_2+\theta_3) \\ L_3\sin(\theta_2+\theta_3) \end{bmatrix}. \tag{7}$$

d) End–effector position: Summing the three link contributions, the spatial forward kinematics of the manipulator is obtained as

$$\mathbf{p} = \begin{bmatrix} L_1\cos\theta_1 + L_2\cos\theta_1\cos\theta_2 + L_3\cos\theta_1\cos(\theta_2+\theta_3) \\ L_1\sin\theta_1 + L_2\sin\theta_1\cos\theta_2 + L_3\sin\theta_1\cos(\theta_2+\theta_3) \\ L_2\sin\theta_2 + L_3\sin(\theta_2+\theta_3) \end{bmatrix}, \tag{8}$$

Jacobian and Differential Kinematics: The joint angular velocities of the 3-DOF spatial robotic arm are related to the linear velocity of the end-effector through the Jacobian matrix $J(\theta)$, as shown in (2). The Jacobian facilitates the conversion of forces and torques from the end-effector to the manipulator joints.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J(\theta) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix}. \tag{9}$$

Jacobian derivation: The linear velocity of the end–effector is obtained by differentiating the forward kinematics in (8)

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.

with respect to time:

$$\dot{\mathbf{p}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \mathbf{J}(\boldsymbol{\theta})\,\dot{\boldsymbol{\theta}}, \qquad \dot{\boldsymbol{\theta}} = \begin{bmatrix} \dot{\theta}_1 & \dot{\theta}_2 & \dot{\theta}_3 \end{bmatrix}^\top. \tag{10}$$

Using the chain rule, each component of the end–effector velocity can be written as

$$\dot{p}_i = \sum_{j=1}^{3} \frac{\partial p_i}{\partial \theta_j}\dot{\theta}_j, \qquad p_i \in \{x, y, z\}, \tag{11}$$

from which the Jacobian entries are defined as

$$J_{ij} = \frac{\partial p_i}{\partial \theta_j}, \qquad i = 1, 2, 3,\ j = 1, 2, 3. \tag{12}$$

e) Partial derivatives of x:

$$\frac{\partial x}{\partial \theta_1} = -\sin\theta_1(L_1 + L_2\cos\theta_2 + L_3\cos(\theta_2+\theta_3)), \tag{13}$$

$$\frac{\partial x}{\partial \theta_2} = -\cos\theta_1(L_2\sin\theta_2 + L_3\sin(\theta_2+\theta_3)), \tag{14}$$

$$\frac{\partial x}{\partial \theta_3} = -L_3\cos\theta_1\sin(\theta_2+\theta_3). \tag{15}$$

f) Partial derivatives of y:

$$\frac{\partial y}{\partial \theta_1} = \cos\theta_1(L_1 + L_2\cos\theta_2 + L_3\cos(\theta_2+\theta_3)), \tag{16}$$

$$\frac{\partial y}{\partial \theta_2} = -\sin\theta_1(L_2\sin\theta_2 + L_3\sin(\theta_2+\theta_3)), \tag{17}$$

$$\frac{\partial y}{\partial \theta_3} = -L_3\sin\theta_1\sin(\theta_2+\theta_3). \tag{18}$$

g) Partial derivatives of z:

$$\frac{\partial z}{\partial \theta_1} = 0, \tag{19}$$

$$\frac{\partial z}{\partial \theta_2} = L_2\cos\theta_2 + L_3\cos(\theta_2+\theta_3), \tag{20}$$

$$\frac{\partial z}{\partial \theta_3} = L_3\cos(\theta_2+\theta_3). \tag{21}$$

h) Spatial Jacobian matrix: Collecting the above terms, the translational Jacobian of the spatial 3–DOF manipulator is given by

Let $s_i = \sin\theta_i, c_i = \cos\theta_i, c_{ij} = \cos(\theta_i+\theta_j), s_{ij} = \sin(\theta_i+\theta_j)$.

$$\mathbf{J}(\boldsymbol{\theta}) = \begin{bmatrix} -s_1(L_1 + L_2c_2 + L_3c_{23}) & -c_1(L_2s_2 + L_3s_{23}) & -L_3c_1s_{23} \\ c_1(L_1 + L_2c_2 + L_3c_{23}) & -s_1(L_2s_2 + L_3s_{23}) & -L_3s_1s_{23} \\ 0 & L_2c_2 + L_3c_{23} & L_3c_{23} \end{bmatrix} \tag{22}$$

This derivation corresponds to the standard procedure for computing the geometric Jacobian of a serial robot manipulator, as described in Sciavicco and Siciliano [14, Sec. 3.2.1]. The Jacobian matrix links joint-space motion(joint velocities) to Cartesian motion(end-effector velocities).

B. Dynamic Modeling

The non-linear dynamics of the 3-DOF robotic manipulator are expressed in compact matrix form using the Euler–Lagrange formulation as

$$M(\theta)\,\ddot{\theta} + C(\theta,\dot{\theta})\,\dot{\theta} + G(\theta) = \tau, \tag{23}$$

where the generalized coordinates, velocities, and joint torques are defined as

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}, \qquad \dot{\theta} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix}, \qquad \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix}. \tag{24}$$

Here, θ_i denotes the i-th joint angle, $\dot{\theta}_i$ the corresponding joint angular velocity, and τ_i the actuated torque at joint i. The vectors θ, $\dot{\theta}$, and $\tau \in \mathbb{R}^3$ describe the joint configuration, velocities, and control inputs of the manipulator. The inertia matrix $M(\theta) \in \mathbb{R}^{3\times3}$ is symmetric positive definite, $C(\theta,\dot{\theta}) \in \mathbb{R}^{3\times3}$ contains Coriolis and centrifugal terms, and $G(\theta) \in \mathbb{R}^3$ is the gravity vector.

Solving (23) for the joint accelerations yields

$$\ddot{\theta} = M(\theta)^{-1}\big(\tau - C(\theta,\dot{\theta})\dot{\theta} - G(\theta)\big). \tag{25}$$

Writing (23) componentwise gives the joint torque equations

$$\begin{aligned} M_{11}\ddot{\theta}_1 + M_{12}\ddot{\theta}_2 + M_{13}\ddot{\theta}_3 + [C(\theta,\dot{\theta})\dot{\theta}]_1 + G_1(\theta) &= \tau_1, \\ M_{21}\ddot{\theta}_1 + M_{22}\ddot{\theta}_2 + M_{23}\ddot{\theta}_3 + [C(\theta,\dot{\theta})\dot{\theta}]_2 + G_2(\theta) &= \tau_2, \\ M_{31}\ddot{\theta}_1 + M_{32}\ddot{\theta}_2 + M_{33}\ddot{\theta}_3 + [C(\theta,\dot{\theta})\dot{\theta}]_3 + G_3(\theta) &= \tau_3. \end{aligned} \tag{26}$$

C. Flow Diagram

The flow diagram in Fig.2 illustrates the sequential steps involved in trajectory generation, kinematic computation, and control torque calculation for the robotic manipulator.

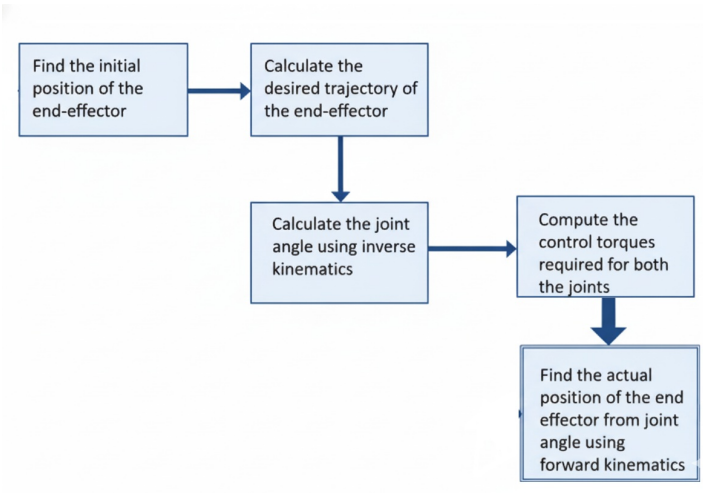


Fig. 2: Flow diagram of the modeling and simulation process

III. Methodology

This work adopts a stepwise approach to evaluate two different control strategies—Proportional-Integral-Derivative (PID) and Model Predictive Control (MPC)—for a 3-DOF robotic arm. In Phase I, PID controller is implemented and tested for each joint to establish a baseline performance level. In Phase II, a Model Predictive Control (MPC) framework is implemented to address the nonlinearities and coupling effects that limit the effectiveness of PID control.

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.

Both phases are validated through MATLAB/Simulink simulations, with the main performance criteria being trajectory tracking accuracy, robustness, and adaptability to system dynamics.

A. Phase I: PID Control of the 3-DOF Arm

The PID controller works on the classical feedback control strategy and generates control actions based on the system error. The control law for the PID controller is formulated as in (20).

$$\tau_i(t) = K_{p,i} e_i(t) + K_{i,i} \int e_i(t)\, dt + K_{d,i} \frac{de_i(t)}{dt}. \tag{27}$$

where $\tau_i(t)$ is the control torque applied to the ith joint, $e_i(t) = \theta_{i,\text{ref}}(t) - \theta_i(t)$ is the tracking error between the reference and actual joint angles, $K_{p,i}$, $K_{i,i}$, and $K_{d,i}$ are the proportional, integral, and derivative gains for the ith joint, respectively, $\int e_i(t)\, dt$ represents the accumulated error over time, used to eliminate steady-state offset, $\frac{de_i(t)}{dt}$ denotes the rate of change of error, which improves transient response and damping.

Although the 3-DOF robotic arm exhibits highly nonlinear dynamics due to Coriolis/centrifugal effects, gravity, configuration-dependent inertia, and coupling between joints, the PID controller is able to operate on the nonlinear system because it is implemented in the joint space and uses instantaneous feedback of the joint angles and velocities. Instead of canceling or linearizing the nonlinear dynamics; instead, the control system automatically compensates for these effects through continuous error correction.

The nonlinearities are explicitly captured by the dynamic model given by equation (16) and the PID torque command acts as the external input driving this nonlinear system. At each time step, the full nonlinear dynamics are evaluated based on the current joint state. Thus, any change in robot configuration directly modifies the system response, and the PID controller compensates by adjusting the torque input according to the instantaneous tracking error.

Finally, despite the nonlinear robot dynamics, the PID controller remains effective because it continuously receives updated joint angle and velocity measurements, allowing it to react to all nonlinear behaviors—including mass variations, gravity coupling, and velocity-dependent Coriolis forces. While PID does not explicitly cancel these nonlinearities (as a computed-torque or feedback-linearizing controller would), its feedback structure and properly tuned gains ensure acceptable tracking performance for the trajectory used in this work.

B. Phase II: MPC Control of the 3-DOF Arm

Model Predictive Control (MPC) is employed to generate optimal joint torques that drive the end-effector along a desired trajectory while respecting the nonlinear dynamics of the manipulator. At each sampling instant, the controller predicts the system evolution over a finite horizon and solves an optimization problem to determine the control inputs that minimize a cost function. Only the first input of the optimized sequence is applied, following the receding-horizon strategy.

Nonlinear Dynamics of the Manipulator: The state of the robot is defined as

$$x = \begin{bmatrix} \theta^\top & \dot{\theta}^\top \end{bmatrix}^\top, \tag{28}$$

and the continuous-time rigid-body dynamics of the 3-DOF arm are given by

$$\dot{x} = f(x, \tau) = \begin{bmatrix} \dot{\theta} \\ M^{-1}(\theta)\big[\tau - C(\theta, \dot{\theta})\dot{\theta} - G(\theta)\big] \end{bmatrix}, \qquad x = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}. \tag{29}$$

Runge–Kutta (RK4) Discretization for MPC Prediction: Following common practice in NMPC for robotic systems, the continuous nonlinear dynamics are numerically integrated using a fixed-step RK4 scheme with sampling period T_s. RK-based propagation provides high prediction accuracy without explicit linearization and is computationally efficient when paired with RTI/SQP solvers [15], [16]. For the state x_k and torque input τ_k, the RK4 stages are

$$\begin{aligned} k_1 &= f(x_k, \tau_k), \\ k_2 &= f\big(x_k + \tfrac{T_s}{2} k_1,\ \tau_k\big), \\ k_3 &= f\big(x_k + \tfrac{T_s}{2} k_2,\ \tau_k\big), \\ k_4 &= f(x_k + T_s k_3,\ \tau_k). \end{aligned} \tag{30}$$

and the discrete-time update is

$$x_{k+1} = x_k + \frac{T_s}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right), \tag{31}$$

where x_k and x_{k+1} denote the state vectors at time steps k and $k+1$, respectively; τ_k is the control torque applied at step k; $f(x, \tau)$ represents the nonlinear system dynamics; and T_s is the sampling period. The terms k_1, k_2, k_3, and k_4 correspond to the slope evaluations in the Runge–Kutta integration scheme.

The RK4 discretization induces the discrete-time mapping

$$x_{k+1} = f_d(x_k, \tau_k), \tag{32}$$

which enables accurate nonlinear state propagation for MPC without linearization. Using this mapping, the predicted states over a horizon N satisfy

$$x_{k+i+1|k} = f_d(x_{k+i|k}, \tau_{k+i}), \qquad i = 0, 1, \ldots, N-1. \tag{33}$$

Cost Function and Optimization: The NMPC cost function is defined as

$$J = \sum_{k=1}^{N} w_k\, e_k^\top Q e_k + \sum_{k=2}^{N} \beta_k\, \dot{e}_k^\top \dot{e}_k + \tau_k^\top R \tau_k, \tag{34}$$

where $e_k = \theta_{d,k} - \theta_k$ denotes the position tracking error and $\dot{e}_k = \dot{\theta}_{d,k} - \dot{\theta}_k$ denotes the velocity tracking error. The matrices Q and R weight tracking accuracy and control effort, while w_k and β_k are time-varying scalar weights that regulate the relative importance of position and velocity errors at each prediction step.

The optimal control sequence is obtained by solving

$$\tau^* = \arg\min_{\tau_{0:N-1}} J(\tau) \quad \text{subject to the nonlinear dynamics.} \tag{35}$$

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.

Numerical Solution: The resulting nonlinear optimization problem is solved online at each sampling instant using MATLAB's `fminunc` solver in a single-shooting framework. The optimizer updates the torque sequence, while RK4 integration is used to propagate the system states. After computing the optimal sequence τ^*, only the first input is applied,

$$\tau_k = \tau_0^*. \tag{36}$$

The system state is then advanced using the RK4 integrator, and the NMPC procedure is repeated at the next sampling instant. This strategy yields accurate tracking performance despite the manipulator's nonlinear and coupled dynamics.

IV. Simulation and Results

Simulations are carried out in MATLAB/Simulink with fourth-order Runge–Kutta (RK4) integration. The robot parameters are defined as follows: the link lengths are $L_1 = 0.8\,\text{m}$, $L_2 = 0.6\,\text{m}$, and $L_3 = 0.4\,\text{m}$, while the corresponding link masses are $m_1 = 1.0\,\text{kg}$, $m_2 = 0.8\,\text{kg}$, and $m_3 = 0.6\,\text{kg}$. The moments of inertia of the links are $I_1 = 0.01$, $I_2 = 0.008$, and $I_3 = 0.006\,\text{kg}\cdot\text{m}^2$. The gravitational acceleration is taken as $g = 9.81\,\text{m/s}^2$. The system is simulated with a sampling time of $T_s = 0.005\,\text{s}$ over a total duration of $T_{\text{sim}} = 5\,\text{s}$. A circular trajectory, as indicated in (26), has been selected for the end-effector.

$$\begin{aligned} x_{\text{ref}}(t) &= 0.5 + 0.3\cos(\omega t), \\ y_{\text{ref}}(t) &= 0.5 + 0.3\sin(\omega t), \\ z_{\text{ref}}(t) &= 0.5. \end{aligned} \tag{37}$$

A. Phase I – PID Control Results for 3-DOF Arm

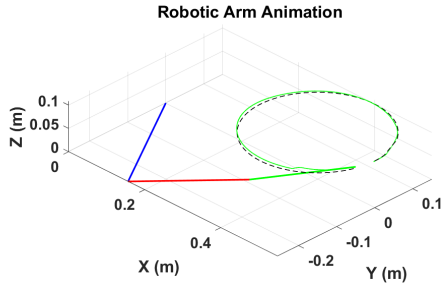


Fig. 3: Trajectory tracking of the 3-DOF arm using a PID controller

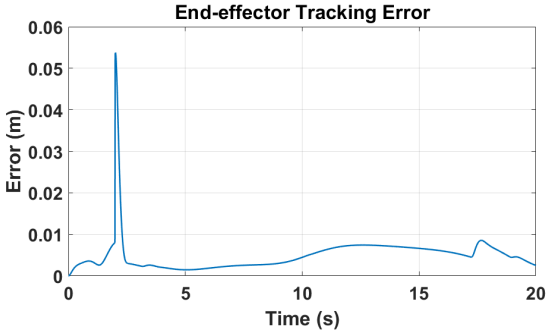


Fig. 4: Tracking error of the 3-DOF arm using a PID controller

The controller gains are obtained through a combination of analytical estimation and fine-tuning via simulation. Initially, K_p, K_i, and K_d values are chosen using the classical Ziegler–Nichols tuning approach based on the system's step response characteristics. The open-loop response of each joint is analyzed to determine the ultimate gain and period, which provides a baseline for proportional, integral, and derivative terms. Subsequently, these gains are iteratively refined in MATLAB/Simulink to achieve satisfactory transient and steady-state performance. The tuning objectives are to minimize the steady-state tracking error for joint positions, achieve a critically damped or slightly underdamped transient response, and reduce overshoot. For the 3-DOF system, the final tuned values are selected as:

$$K_p = \begin{pmatrix} 100 & 0 & 0 \\ 0 & 80 & 0 \\ 0 & 0 & 60 \end{pmatrix},\ K_i = \begin{pmatrix} 10 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 6 \end{pmatrix},\ K_d = \begin{pmatrix} 20 & 0 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 12 \end{pmatrix} \tag{38}$$

The proportional term dominates the transient response, while the integral term eliminates residual steady-state error, and the derivative term improves damping during rapid trajectory changes. Figure 3 illustrates the trajectory tracking performance of the robotic arm with a PID controller, while Fig. 4 shows the corresponding tracking error. From the plots, it is observed that the PID controller provides reasonable tracking under nominal conditions. However, steady-state errors remain noticeable, and the controller response shows small overshoots. Eventhough PID provides a useful baseline, it does not fully address the nonlinear and coupled nature of a 3-DOF arm.

B. Phase II – MPC Control Results for 3-DOF Arm

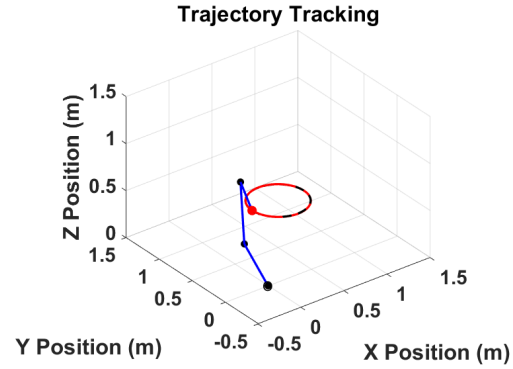


Fig. 5: Trajectory tracking of 3-DOF arm using MPC

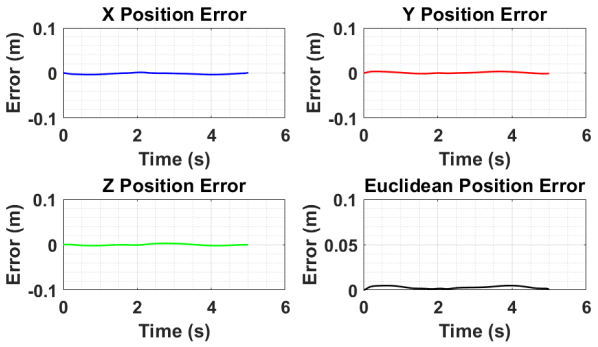


Fig. 6: Position Error – MPC for 3-DOF

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.

The simulation of the 3-DOF robotic arm using Model Predictive Control is carried out with the following configuration. The MPC directly outputs the joint torques τ_1, τ_2, and τ_3 at each sampling instant, optimizing a quadratic cost function over a finite prediction horizon. The state weight matrix is $Q = \mathrm{diag}([100,\ 100,\ 100])$, which penalizes tracking errors in the end-effector position (x, y, z). The control weight matrix is $R = \mathrm{diag}([0.1,\ 0.1,\ 0.1])$, which penalizes large torque inputs to ensure smooth and efficient actuation. The sampling period T_s is 0.01 s. Prediction horizon, N_p is 20, and the preview window for reference trajectory, N_{preview} is 35. The optimization is carried out using the `fminunc` solver configured with a Sequential Quadratic Programming (SQP) approach. This method updates the control inputs by iteratively solving a locally quadratic approximation of the cost function. The solver is allowed up to 200 iterations, with an optimality tolerance of 10^{-6} and a step tolerance of 10^{-6}, ensuring that the optimization terminates only when further improvements in the solution become negligibly small. At each sampling instant, the MPC minimizes the cost function given in equation (16) with nonlinear dynamics enforced through the RK4 prediction model. Figures 5 to 6 present the results obtained with MPC. The MPC controller successfully tracks the 3D trajectory with high precision, exhibiting errors within a few millimeters, often about 0.01 m.The generated control inputs are smoother compared to PID, which indicates reduced mechanical stress on the joints.

C. Comparative Analysis

A comparison of the two controllers shows clear differences in performance. PID works reasonably well for simple trajectories, but it tends to produce steady-state errors and oscillations, especially for circular motion. In contrast, MPC achieves more accurate tracking by predicting future system behavior and adjusting control inputs accordingly. PID also requires careful tuning of K_p, K_i, and K_d gains and still exhibits oscillatory responses, whereas MPC provides smoother and more stable error behavior.

In terms of control effort, PID often results in abrupt torque changes, while MPC generates smoother inputs. Although MPC involves higher computational effort due to online optimization, modern solvers make real-time implementation practical. Overall, PID is simple and easy to implement, but its performance is limited for nonlinear multi-DOF systems, whereas MPC offers better accuracy, robustness, and smoother control.

V. Conclusion

In this work, the control of a 3-DOF robotic arm is investigated using both PID and MPC strategies. The PID controller demonstrates its traditional strengths—simplicity, ease of implementation, and low computational demand—which make it suitable for basic trajectory tracking in linear or mildly nonlinear systems. But ts reactive nature prevents it from anticipating future errors, and it lacks mechanisms to handle joint limits, torque constraints, and nonlinear coupling between joints. Consequently, PID performance degrades when faced with multivariable interactions, strong nonlinearities, or rapid dynamic behavior, resulting in poorer tracking accuracy compared to more advanced control approaches.

In contrast, the MPC framework offers significantly improved performance for the 3-DOF robotic arm. By predicting future system behavior over a finite horizon, MPC generates anticipatory control inputs that effectively manage nonlinear dynamics, joint interactions, and actuator limitations. This predictive capability enables smoother motion and lower steady-state tracking error across the X, Y, and Z axes. Although MPC requires an accurate dynamic model and incurs higher computational cost due to the need for online optimization at each sampling step, its ability to provide robust, high-precision trajectory tracking makes it a more suitable and powerful control strategy for complex robotic applications.

As a next step, the proposed work can be extended to the control of multi-agent robotic arms, involving collaborative path planning, coordination, and collision avoidance.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2010.

[2] S. B. Niku, *Introduction to Robotics: Analysis, Control, Applications*, 4th ed., Wiley, 2020.

[3] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, John Wiley & Sons, 2006.

[4] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2005.

[5] K. H. Ang, G. Chong, and Y. Li, "PID control system analysis, design, and technology," *IEEE Transactions on Control Systems Technology*, vol. 13, no. 4, pp. 559–576, 2005.

[6] A. Eswehli and I. Alsogkier, "Trajectory Tracking Control of Two-Link Robotic Manipulator Using PID and MPC Controllers," *Journal of Academic Research (Applied Sciences)*, vol. 28, pp. 28–41, 2024.

[7] E.-H. Guechi, S. Bouzoualegh, Y. Zennir, and S. Blažič, "MPC Control and LQ Optimal Control of a Two-Link Robot Arm: A Comparative Study," *Machines*, vol. 6, no. 4, p. 51, 2018.

[8] M. Bikova, D. Stavrov, V. O. Latkoska, and B. Hristov, "Model Predictive Control of a 2-DOF Robotic Arm and Comparative Analysis with Fuzzy Logic Control," in *Proc. XVI International Conference ETAI*, 2024.

[9] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained Model Predictive Control: Stability and Optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[10] D. Q. Mayne, "Model Predictive Control: Recent Developments and Future Promise," *Automatica*, vol. 50, no. 12, pp. 2967–2986, 2014.

[11] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, Pearson Education, 2005.

[12] Y. Kuwata, S. Karaman, E. Frazzoli, and J. How, "Real-Time Motion Planning With Applications to Autonomous Urban Driving," in *AIAA Guidance, Navigation, and Control Conference*, 2005.

[13] E. F. Camacho and C. Bordons, *Model Predictive Control*, Springer, 2004.

[14] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.

[15] O. Y. Ismael, M. Almaged, and A. I. Abdulla, "Nonlinear Model Predictive Control-based Collision Avoidance for Mobile Robot," *Journal of Robotics and Control*, vol. 5, no. 2, pp. 220–230, 2024.

[16] J. Frey, "Gauss–Newton Runge–Kutta Integration for Efficient NMPC Hessian Approximation," in *Proc. European Control Conference (ECC)*, 2024.

[17] R. Resmi and J. Jacob, "Distributed Model-Predictive Control Using Exponentially Weighted Laguerre Functions and Periodic Time Triggering for Dynamic Consensus of Linear Multiagent Systems," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 9, pp. 9867–9876, 2023.

[18] R. Resmi, S. J. Mija, and J. Jacob, "Discrete Laguerre-based Model Predictive Control for Dynamic Consensus of a Vehicle Platoon with Time Delay," *International Journal of Systems Science*, vol. 53, no. 12, pp. 2566–2583, 2022.

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on October 29,2026 at 19:32:22 UTC from IEEE Xplore. Restrictions apply.